MyCodeSchool

Linux — exerciții

Docker, deploy pe AWS și nginx · 147 exerciții în 14 module + proiectul final · modulele sunt aceleași ca în `TEORIE.pdf`

Pentru fiecare exercițiu: scrii comanda (o găsești în modulul cu același număr din `TEORIE.pdf`), compari ce apare pe ecran cu coloana verde și bifezi ❑. Nu treci mai departe cu un rezultat diferit. `…` = o parte care diferă de la server la server.

Terenul de antrenament (îl lipești pe server la modulul 3)

Face `~/antrenament/` cu loguri, configurări, un CSV și fișiere vechi, mereu aceleași, ca rezultatele să fie cele din fișă. O iei de la capăt cu `rm -r ~/antrenament` și blocul din nou.

```
cd ~ && mkdir -p antrenament/{logs,config,date,vechi,proiect/src,proiect/test,proiect/docs}
cd ~/antrenament
awk 'function r(n) { x = (x * 75 + 74) % 65537; return x % n }
BEGIN { x = 42
  split("/api/users /api/products /api/orders /login /swagger-ui.html /api/courses", u, " ")
  split("200 200 200 200 201 302 404 404 500 200", c, " ")
  for (i = 1; i <= 500; i++) {
    p = u[r(6) + 1]; m = ((p == "/login" || p == "/api/orders") && r(3) == 0) ? "POST" : "GET"
    printf "10.0.0.%d - - [06/Nov/2026:%02d:%02d:%02d +0200] \"%s %s HTTP/1.1\" %s %d\n",
      int(sqrt(r(529))) + 1, 8 + int(i / 60), i % 60, r(60), m, p, c[r(10) + 1], r(5000) + 100
  } }' > logs/access.log
awk 'function r(n) { x = (x * 75 + 74) % 65537; return x % n }
BEGIN { x = 7
  split("INFO INFO INFO INFO WARN WARN ERROR DEBUG INFO INFO", n, " ")
  split("UserService OrderService PaymentService AuthService", s, " ")
  for (i = 1; i <= 300; i++)
    printf "2026-11-06 %02d:%02d:%02d %-5s [%s] request %d took %d ms\n",
      9 + int(i / 60), i % 60, r(60), n[r(10) + 1], s[r(4) + 1], 1000 + i, r(900) + 10
}' > logs/app.log
printf '%s\n' '# configurare' 'server.port=8080' 'db.host=localhost' 'db.port=3306' \
  'db.user=root' '# db.password=secret' 'log.level=INFO' 'cache.enabled=true' > config/app.conf
printf 'DB_PASSWORD=root\nJWT_SECRET=schimba-ma\n' > config/.env
printf '%s\n' nume,oras,varsta,nota Ana,Cluj,19,9 Bogdan,Iasi,22,7 Carmen,Cluj,20,10 \
  Dan,Bucuresti,21,6 Elena,Iasi,19,8 Florin,Cluj,23,5 Gabi,Bucuresti,20,9 \
  Horia,Timisoara,22,7 Ioana,Cluj,21,10 Mihai,Iasi,20,4 > date/elevi.csv
for n in 1 2 3; do echo "nota $n" > proiect/src/nota$n.txt; done
echo "TODO: scrie testele" > proiect/test/todo.txt
echo "TODO: documentatie" > proiect/README.md
fallocate -l 8M date/mare.bin
for n in 1 2 3; do fallocate -l ${n}M vechi/backup-$n.bak; done
touch -d "10 days ago" vechi/*
```

1. Serverul și sistemul de operare

Pe server (`ssh devops` sau cu cheia).

❑	1	Afișezi numele și versiunea kernelului.	`Linux 6.…-aws`
❑	2	Afișezi distribuția și versiunea ei.	`Ubuntu 24.04… LTS`
❑	3	Afli câte procesoare (vCPU) are serverul.	`2`
❑	4	Afli același număr din `/proc/cpuinfo`, numărând rândurile `processor`.	`2`
❑	5	Afli memoria totală.	`~1.9Gi`
❑	6	Afli arhitectura procesorului.	`x86_64`
❑	7	Afli pe ce tip de mașină virtuală rulezi, cu `systemd-detect-virt`.	`amazon`

2. Sistemul de fișiere

❑	8	Listezi folderele din `/`.	`bin boot dev etc home … tmp usr var`
❑	9	Afli unde e programul `ls`.	`/usr/bin/ls`
❑	10	Afli unde e programul `docker`.	`/usr/bin/docker`
❑	11	Afișezi conținutul lui `/etc/hostname`.	`ip-172-31-…`
❑	12	Listezi doar fișierele ascunse din `~`.	`.bashrc`, `.profile`, `.ssh` …
❑	13	Scrii `test` în `/dev/null`, apoi îl citești.	nimic: ce intră acolo dispare
❑	14	Afli cât ocupă `/var/log`.	o mărime în `M`
❑	15	Afli cât ocupă `/usr` și `/home`, cu o singură comandă.	`/usr` mult mai mare decât `/home`

3. Linia de comandă: navigare și fișiere

Lipești pe server blocul **Terenul de antrenament** de la începutul fișei. De aici încolo pornești din `~/antrenament`, în ordine: unele exerciții schimbă fișierele pentru cele de după.

❑	16	Afișezi calea completă a folderului curent.	`/home/ubuntu/antrenament`
❑	17	Listezi `config` cu fișierele ascunse și cu detalii.	`.env` și `app.conf`, ale lui `ubuntu`
❑	18	Numeri toate fișierele (nu folderele), inclusiv ascunse.	`14`

❑ 19	Numeri folderele, fără folderul curent.	`8`
❑ 20	Creezi cu o singură comandă `proiect/docs/img` și `proiect/docs/text`.	`find proiect/docs -type d`: 3 rânduri
❑ 21	Copiezi `config/app.conf` în `proiect/docs/text/` cu numele `app.conf.bak`.	`app.conf.bak` în `text`
❑ 22	Muți `proiect/README.md` în `proiect/docs/`.	în `proiect`: `docs src test`
❑ 23	Ștergi `proiect/docs/img` cu comanda pentru foldere goale.	în `proiect/docs`: `README.md text`
❑ 24	Copiezi tot `proiect` în `/tmp/proiect-copie`, apoi ștergi copia.	`ls /tmp` nu o mai arată
❑ 25	Primul rând din `logs/access.log`.	`10.0.0.7 … "GET /api/orders HTTP/1.1" 500 4433`
❑ 26	Ultimele 2 rânduri din `logs/app.log`.	ultimul: `… request 1300 took 844 ms`
❑ 27	Rândurile 100–105 din `access.log`, cu `head` și `tail`.	6 rânduri, primul `10.0.0.20 … /api/products … 404 4109`
❑ 28	Câte rânduri au cele două loguri, cu o singură comandă.	`500`, `300`, `800 total`
❑ 29	Câte cuvinte are `config/app.conf`.	`10`
❑ 30	`date/elevi.csv` cu rândurile numerotate.	11 rânduri, ultimul `11 Mihai,Iasi,20,4`

4. Instalarea programelor: apt

❑ 31	Actualizezi lista de pachete.	la final `Reading package lists… Done`
❑ 32	Afli versiunea pachetului `tree`, fără să-l instalezi.	`Version: 2.1.1…`
❑ 33	Instalezi `tree` și îl folosești pe `~/antrenament`, 1 nivel.	`tree --version` = `tree v2.1.1`
❑ 34	Verifici dacă `jq` e instalat.	nu îl găsește
❑ 35	Instalezi `jq` și scoți valoarea lui `a` din `{"a": 1}`.	`1`
❑ 36	Dezinstalezi `jq` și verifici.	`which jq` nu afișează nimic
❑ 37	Afli din ce repository vine pachetul `docker-ce`.	`https://download.docker.com/linux/ubuntu …`

5. Editorul vim

❑ 38	Creezi cu vim `~/antrenament/vim.txt` cu 5 rânduri: `unu`, `doi`, `trei`, `patru`, `cinci`; salvezi și ieși.	`cat`: cele 5 rânduri
❑ 39	Deschizi din nou, ștergi rândul `trei` cu o comandă vim, salvezi.	`unu doi patru cinci`
❑ 40	Înlocuiești în tot fișierul `u` cu `U`, cu o singură comandă vim.	`UnU doi patrU cinci`
❑ 41	Copiezi primul rând la finalul fișierului, cu comenzi vim.	ultimul rând `UnU`
❑ 42	Schimbi ceva, apoi ieși fără să salvezi.	fișierul rămâne cum era la ex. anterior

6. Utilizatori și grupuri

Pe server, cu `sudo` unde e nevoie.

❑ 43	Afișezi id-ul tău și grupurile.	`uid=1000(ubuntu)`, printre grupuri `sudo` și `docker`
❑ 44	Afișezi rândul tău din `/etc/passwd` și numești fiecare câmp.	`ubuntu:x:1000:1000:Ubuntu:/home/ubuntu:/bin/bash`
❑ 45	Creezi grupul `devops`.	`getent group devops` → `devops:x:100…:`
❑ 46	Creezi utilizatorul `ana`, cu folder acasă și shell `/bin/bash`.	există `/home/ana`
❑ 47	O pui pe `ana` în grupul `devops`, fără să-i scoți celelalte grupuri.	`id ana` conține `(devops)`
❑ 48	Îi setezi o parolă.	`password updated successfully`
❑ 49	Devii `ana`; afișezi cine ești și unde ești.	`ana`, `/home/ana`
❑ 50	Ca `ana`, încerci `sudo apt update`.	`ana is not in the sudoers file`
❑ 51	Ca `ana`, încerci să citești `/home/ubuntu/antrenament/config/app.conf`.	`Permission denied`
❑ 52	Revii la `ubuntu`, ștergi `ana` cu tot cu folderul ei, apoi grupul `devops`.	`getent passwd ana` și `getent group devops` nu afișează nimic

7. Permisiuni

Din nou în `~/antrenament`.

❑ 53	`config/.env` îl poți citi și scrie doar tu (cu cifre).	`-rw-------`
❑ 54	`proiect`: tu ai tot, grupul citește și intră, ceilalți nimic (cu cifre).	`drwxr-x---`
❑ 55	`date/elevi.csv`: întâi `640`, apoi adaugi scriere grupului cu `+`.	`-rw-rw----`
❑ 56	Scriptul `salut.sh` cu `echo salut`; îi setezi cu `=`: proprietar rwx, grup rx, ceilalți nimic; îl rulezi.	`-rwxr-x---` și `salut`
❑ 57	Scoți dreptul de scriere pentru grup și ceilalți pe tot `antrenament`, recursiv.	`logs/app.log`: `-rw-r--r--`

❑ 58	Cine e proprietarul lui `/etc/shadow` și cine îl poate citi.	`-rw-r----- root shadow`
❑ 59	Faci `/tmp/al-lui-root.txt` al lui `root` și încerci să scrii în el.	`Permission denied`

8. Pipe-uri și redirecționări

❑ 60	`ls config nu-exista`: ce merge în `ok.txt`, eroarea în `err.txt`.	în `err.txt`: `cannot access`
❑ 61	Aceeași comandă, totul în `tot.txt`.	`wc -l tot.txt` = `3`
❑ 62	Aceeași comandă, erorile aruncate.	doar conținutul lui config, fără eroare
❑ 63	Numeri rândurile din `app.log` dându-l ca intrare cu `<`.	`300`
❑ 64	Scrii `primul` în `linii.txt`, apoi adaugi `al doilea`.	`cat`: 2 rânduri
❑ 65	Scrii din nou `primul` cu `>`.	`cat`: 1 rând
❑ 66	Rulezi `ls /nu-exista` și afișezi codul de ieșire, apoi la `ls /`.	`2`, apoi `0`

9. Căutat și prelucrat text

În `elevi.csv`: `nume,oras,varsta,nota`. În `access.log`: IP = `$1`, adresa = `$7`, codul = `$9`, octeții = `$10`.

❑ 67	Câte rânduri `ERROR` are `app.log`.	`26`
❑ 68	Câte cereri au primit codul 500.	`59`
❑ 69	`app.conf` fără comentarii; câte rânduri rămân.	`6`
❑ 70	Câte cereri `POST` în total; câte pe `/login`.	`61`; `29`
❑ 71	Câte rânduri `WARN` sau `ERROR` are `PaymentService`.	`21`
❑ 72	Doar numele fișierelor care conțin `TODO`, în tot folderul.	`proiect/test/todo.txt`, `proiect/docs/README.md`
❑ 73	Câte rânduri conțin „error”, fără litere mari, în fiecare fișier din `logs`.	`access.log:0`, `app.log:26`
❑ 74	Primele 3 rânduri `WARN/ERROR` din `app.log`, cu numărul rândului.	rândurile `8`, `9`, `12`
❑ 75	Doar valoarea lui `db.port`.	`3306`
❑ 76	Numele elevilor, invers alfabetic, primele 3.	`Mihai`, `Ioana`, `Horia`
❑ 77	Orașele, fiecare o singură dată.	`Bucuresti Cluj Iasi Timisoara`
❑ 78	Câți elevi are fiecare oraș, descrescător.	`4 Cluj`, `3 Iasi`, `2 Bucuresti`, `1 Timisoara`
❑ 79	Elevii cu nota 10.	`Carmen`, `Ioana`
❑ 80	Media notelor.	`7.5`
❑ 81	Elevii mai mari de 20 de ani.	5: `Bogdan Dan Florin Horia Ioana`
❑ 82	Top 3 IP-uri după numărul de cereri, cu numărul lor.	`55 10.0.0.23`, `46 10.0.0.22`, `44 10.0.0.21`
❑ 83	Câte IP-uri diferite au trimis cereri.	`23`
❑ 84	Fiecare cod HTTP cu numărul lui, descrescător.	`247 200`, `100 404`, `59 500`, `51 302`, `43 201`
❑ 85	Cea mai cerută adresă.	`90 /api/users`
❑ 86	Totalul octeților trimiși pe `/api/orders`.	`246643`
❑ 87	Câte cereri au venit în fiecare oră.	`08`: 59, `09`–`15`: câte 60, `16`: 21
❑ 88	Durata medie (ms) la `OrderService`, fără zecimale.	`456`
❑ 89	Câte rânduri are fiecare nivel din `app.log`, descrescător.	`182 INFO`, `51 WARN`, `41 DEBUG`, `26 ERROR`
❑ 90	IP-ul care a primit cele mai multe răspunsuri 404.	`12 10.0.0.21`
❑ 91	Câte fișiere `.txt` sunt.	`9`
❑ 92	Fișierele mai mari de 1 MB.	`mare.bin`, `backup-2.bak`, `backup-3.bak`
❑ 93	Cele mai mari 3 fișiere, cu mărimea, de la mic la mare.	`2.0M`, `3.0M`, `8.0M date/mare.bin`
❑ 94	Fișierele modificate acum mai mult de 7 zile.	cele 3 `backup-*.bak`
❑ 95	Șterge cu `find`, într-o comandă, `.bak`-urile mai vechi de 7 zile; câte `.bak` rămân și care.	`1`: `proiect/docs/text/app.conf.bak`
❑ 96	Folderele goale.	`./vechi`
❑ 97	Fișierele ascunse.	`./config/.env`
❑ 98	Afișezi `app.conf` cu `localhost` → `db`, fără să schimbi fișierul.	`db.host=db` pe ecran, `db.host=localhost` în fișier
❑ 99	Schimbi în fișier `log.level=INFO` în `DEBUG`.	`log.level=DEBUG`
❑ 100	Rândurile 5–8 din `app.log`, cu `sed`.	4 rânduri, `request 1005` … `1008`
❑ 101	Faci `date/elevi-pv.csv`: aceleași date, cu `;` în loc de `,`.	primul rând `nume;oras;varsta;nota`

❑ 102	Șterge din `app.conf`, în fișier, comentariile.	`wc -l = 6`

10. Scripturi bash

Scripturile le faci în `~/antrenament` și le rulezi cu `./nume.sh`.

❑ 103	`nivel.sh`: primește un nivel (`$1`) și afișează câte rânduri are în `app.log`.	`./nivel.sh WARN` → `51`; `ERROR` → `26`
❑ 104	`nivel.sh` fără argument afișează `Folosire: ./nivel.sh NIVEL` și iese cu `1`.	mesajul, apoi `echo $?` = `1`
❑ 105	`args.sh` afișează câte argumente a primit și pe toate.	`./args.sh a b c` → `3` și `a b c`
❑ 106	`exista.sh` primește o cale și afișează `fisier`, `folder` sau `lipseste`.	`logs/app.log` → `fisier`, `logs` → `folder`, `nu` → `lipseste`
❑ 107	`servicii.sh`: cu un `for` peste cele 4 servicii, afișează câte rânduri are fiecare în `app.log`.	`UserService 87`, `OrderService 74`, `PaymentService 81`, `AuthService 58`
❑ 108	`note.sh`: cu `while read`, pentru fiecare elev din `elevi.csv` afișează `Ana are nota 9`.	10 rânduri, ultimul `Mihai are nota 4`
❑ 109	`numara.sh`: funcția `numara` primește un cuvânt și întoarce câte rânduri îl conțin; o chemi pentru `INFO` și `DEBUG`.	`182`, `41`
❑ 110	`raport.sh`: pe rânduri, totalul cererilor, câte 500 și IP-ul cu cele mai multe cereri.	`500`, `59`, `10.0.0.23`
❑ 111	`verifica.sh`: pentru porturile `8111 8104 8105` afișează `OK/PICAT` după codul HTTP și iese cu `1` dacă a picat măcar unul.	cu un API oprit: `PICAT` și `echo $? = 1`

11. Variabile de mediu

❑ 112	Afișezi `HOME`, `USER` și `SHELL`.	`/home/ubuntu`, `ubuntu`, `/bin/bash`
❑ 113	Setezi `APP=test` fără `export`, apoi îl citești dintr-un `bash -c` nou.	gol: variabila nu ajunge la procesele copil
❑ 114	Același lucru cu `export APP_ENV=dev`.	`dev`
❑ 115	`conectare.sh` afișează `conectare la $DB_HOST`; îl rulezi cu `DB_HOST=mysql` doar pentru comanda aia.	`conectare la mysql`; după, `echo $DB_HOST` e gol
❑ 116	Afișezi `PATH`, câte un folder pe rând.	printre ele `/usr/local/bin` și `/usr/bin`
❑ 117	Muți `nivel.sh` în `~/bin`, pui `~/bin` în `PATH` din `~/.bashrc` și îl rulezi din `/tmp` doar cu `nivel.sh WARN`.	`51`
❑ 118	Ieși, intri din nou: `APP_ENV` și `nivel.sh`.	`APP_ENV` gol, `nivel.sh WARN` merge

12. Rețea

❑ 119	Adresa privată a serverului și interfața.	`ens5` cu `172.31.…/20`
❑ 120	Câte adrese are o rețea `/20`? Dar `/24` și `/16`?	`4096`; `256`; `65536`
❑ 121	Ruta implicită: prin ce adresă iese serverul spre internet.	`default via 172.31.…1 dev ens5`
❑ 122	Adresa publică a serverului, din terminal.	aceeași ca în consola EC2
❑ 123	Adresa IP a lui `hub.docker.com`.	una sau mai multe adrese IP
❑ 124	Trimiți 3 ping-uri la `google.com`.	`3 packets transmitted, 3 received`
❑ 125	Ce programe ascultă pe porturi TCP și pe ce porturi.	`sshd` pe `22`, `docker-proxy` pe porturile aplicațiilor
❑ 126	Verifici cu `nc` portul `8111` și portul `8112`.	`succeeded`; `Connection refused`
❑ 127	Doar codul HTTP de la `localhost:8111/api/users` și de la `localhost:8112`.	`200`; `000`
❑ 128	În consola EC2, regulile inbound ale security group-ului.	`22` și porturile aplicațiilor, toate cu IP-ul tău `/32`
❑ 129	De pe Mac, `curl` pe un port care NU e în security group.	se blochează, apoi timeout

13. SSH

Pe Mac-ul tău (terminalul din VS Code, zsh), apoi pe server.

❑ 130	Afișezi drepturile cheii `.pem`.	`-r--------`
❑ 131	Alias-ul `devops` în `~/.ssh/config`; intri doar cu `ssh devops`.	promptul `ubuntu@ip-172-31-…`
❑ 132	Găsești serverul în `~/.ssh/known_hosts`.	un rând cu IP-ul și cheia serverului
❑ 133	Rulezi pe server, de pe Mac, fără să intri: discul liber și memoria.	`df` și `free` afișate pe Mac
❑ 134	Generezi cheia `~/.ssh/devops_ed25519` fără parolă.	două fișiere: cheia și `.pub`
❑ 135	Adaugi cheia publică în `authorized_keys` pe server, fără să o ștergi pe cea AWS.	`authorized_keys`: 2 rânduri
❑ 136	Intri cu cheia nouă.	`ubuntu@ip-172-31-…`
❑ 137	Pe server: sunt permise parolele la ssh?	`PasswordAuthentication no`
❑ 138	Aduci cu `scp` `raport.sh` de pe server pe Mac, îl trimiți înapoi ca `~/raport2.sh` și îl rulezi prin `ssh devops`.	`500`, `59`, `10.0.0.23`

❏ 139	Scoți cheia ed25519 din authorized_keys și dovedești că nu mai merge.	`Permission denied (publickey)`; `ssh devops` merge

14. Procese, servicii, loguri, programări

Cu aplicațiile pornite: `cd ~/devops-initiere && docker compose up -d`.

❏ 140	Pornești `sleep 1000` în fundal, îi afli PID-ul după nume și îl oprești.	după `kill`, `pgrep sleep` nu afișează nimic
❏ 141	Primele 3 procese după memorie.	procese `java`
❏ 142	Starea serviciilor `docker` și `ssh`.	`active (running)`
❏ 143	Ultimele 10 rânduri de log ale serviciului `docker`.	rânduri cu data de azi
❏ 144	Ultimele 5 rânduri de log ale containerului `online-shop-api`.	loguri Spring
❏ 145	Sesiunea tmux `loguri` cu `tail -f logs/app.log`; detach, ieși de pe server, intri, revii.	logul unde l-ai lăsat
❏ 146	Fusul orar: România.	`EET`
❏ 147	`raport.sh` din cron la fiecare minut în `/home/ubuntu/raport.log`; după 2 minute verifici, apoi ștergi programarea.	2 rapoarte, apoi `no crontab for ubuntu`

15. Proiect final: un script de raport, rulat de la distanță

- ❏ Scrii pe Mac `info-server.sh`: afișează pe rânduri utilizatorul, IP-ul public, memoria liberă, spațiul liber pe `/` și câte containere rulează.
- ❏ Scriptul iese cu codul `1` și mesajul `Docker nu rulează` dacă `docker` nu răspunde.
- ❏ Îl copiezi cu `scp` pe server, în `~/bin`, și îl faci executabil.
- ❏ Îl rulezi de pe Mac, prin `ssh devops`, fără să intri pe server, și salvezi rezultatul local în `info.txt`.
- ❏ Îl programezi cu cron la 5 minute, cu ieșirea și erorile în `/home/ubuntu/info.log`.
- ❏ După 10 minute descarci `info.log` cu FileZilla: are cel puțin 2 rapoarte.
- ❏ Oprești un API, aștepți următorul raport și arăți în `info.log` că s-a schimbat numărul de containere.
- ❏ Faci curat: ștergi programarea cron, `docker compose down`, ieși din tmux, **Stop** la instanță în consola EC2.